MSc Project 1: Time-Series-Enhanced Predictive Process Monitoring

Phase Review 1

Michał Durkalec, Konstantinos Sakkas, and Roman Ležovič

Supervised Process Intelligence (SPI) Lab,
RWTH Aachen University, Aachen, Germany
office@pads.rwth-aachen.de

Abstract. This document review covers Sprint 1 of the project, which includes the objectives, technical implementation, and retrospective. The main achievement was the construction of a fully functional, modular data pipeline that processes raw XES data, splits it into temporal segments, and extracts features for the BPI Challenge 2017 dataset. We successfully trained and evaluated initial baseline models for both classification and regression tasks using classic event-log features. We evaluated linear, tree-based, and gradient boosting algorithms and overcame early performance bottlenecks by optimizing our prefix generation implementation. This pipeline serves as the foundation for integrating external time-series data and further developments in the next sprint.

Keywords: Predictive Process Monitoring · Process Mining · Baseline Models · Event Log Preprocessing · Feature Extraction

1 Introduction & Objectives

1.1 Sprint Goals

Sprint 1 goal was to first establish the basic pipeline that builds the foundational software infrastructure to handle the BPI Challenge 2017 event log [1]. This pipeline includes data ingestion, cleaning, strict temporal splitting, and case prefix generation. Afterwards, we shifted our focus on developing the baseline models that implement our initial predictive models using only classic event-log features (control-flow, temporal, and case attributes). We explicitly excluded time-series data at this stage so baseline models addressed classifying the final result of the loan application and estimating the continuous time left until case completion.

1.2 Team Organization

Building upon the methodologies defined during the project setup, we distributed the tasks accordingly to maximize parallelism. Each stage was tracked as an issue

link on a branch, while each task was tracked as a sub-issue linked to each stage. The project commenced with Michal implementing the foundation pipeline wireframe. Subsequently, he set up the dataset loading, which laid the groundwork for Konstantinos and Roman to handle the preprocessing and feature evaluation. Konstantinos took charge of the cleaning and splitting tasks, while Roman focused on building the trace prefixes and extracting log-based features. The remaining tasks of predicting the outcome and the remaining time were completed by Konstantinos and Michal.

2 Technical Implementation

Sprint 1 produced a modular, end-to-end pipeline covering all stages from raw event log ingestion to baseline model evaluation. The implementation prioritises replaceability: each pipeline stage is an independently swappable component, meaning time-series features can be integrated in Sprint 2 without restructuring existing code. The following subsections describe the architectural pattern that enforces this, the incremental delivery of the seven implementation steps, and the baseline models evaluated against the BPI Challenge 2017 dataset [1].

2.1 Strategy Design Pattern

The pipeline follows a *strategy* design pattern: each stage is a typed callable that accepts and returns one of the domain objects defined in `data/schemas.py` and `data/types.py`. The type aliases for the strategies are:

- **Preprocessor:** `(RawData) -> EventLog`
  — cleans the raw event log.
- **Splitter:** `(EventLog) -> PreprocessedData`
  — temporal train/test split and prefix generation.
- **FeatureExtractor:** `(PreprocessedData) -> FeatureSet`
  — builds labelled feature matrices.
- **Evaluator:** `(ModelArtifact, FeatureSet) -> EvaluationReport`
  — computes metrics.
- **Reporter:** `(ModelArtifact, EvaluationReport, Path | None) -> None`
  — outputs artefacts.

2.2 Baseline Pipeline Overview

The baseline implementation was developed in seven incremental steps, each delivered as a separate pull request. Together they form a fully operational preprocessing-to-evaluation pipeline, laying the foundation onto which time-series features will be grafted in Sprint 2. The pipeline begins with raw XES ingestion and progresses through cleaning, temporal splitting, prefix construction, feature extraction, model training, and evaluation. We reviewed and merged each step independently, keeping the `dev` branch in a consistently working state throughout the sprint.

Table 1 lists the main implementation steps in chronological order, mapping each to the corresponding pull request and the source modules it introduced or extended.

Table 1. Baseline implementation milestones.

Step	Description	Key modules
1	Dataset management (PR #30)	`data/dataset.py`
2	Pipeline wireframe (PR #33)	`pipeline/pipeline.py`, `pipeline/builder.py`
3	Strategy refactor (PR #42)	`data/types.py`, `pipeline/pipeline.py`
4	Prefix generation (PR #43)	`preprocessing/preprocess.py`
5	Event-log preprocessing (PR #47)	`preprocessing/preprocess.py`
6	Log-based features (PR #48)	`features/log_based_features.py`, `features/extraction.py`
7	Baseline models (PR #39-40)	`evaluation/metrics.py`, `models/trainer.py`, `pipeline/checkpointing.py`, `notebooks/BaselineRegression.ipynb`, `notebooks/BaselineClassification.ipynb`

2.3 Models & Metrics

On the baseline notebook (`notebooks/ClassificationBaseline.ipynb`), where we attempt to predict the loan's approval based on the event logs prefixes, we employ three models. The first model is logistic regression, which serves as a simple and interpretable baseline. It estimates the probability of the positive class using a logistic function. The second model is the random forest, which captures the non-linear relationships and interactions between features. Finally, we use the HistGradientBoosting model, which builds decision trees sequentially to correct the errors of previous trees.

The metrics we employ to assess these models are AUC, Brier Score (which quantifies the accuracy of predicted probabilities in relation to actual outcomes), and Accuracy.

On the other baseline notebook (`notebooks/RegressionBaseline.ipynb`), where our objective is to predict the continuous remaining time (in hours) until a case completes, we also employ three models. The first model is Ridge Regression, which assumes a linear relationship between the log features and the remaining time. We then continue using the random forest and HistGradientBoosting models, as mentioned earlier.

We evaluate these models using MAE (Average prediction error in hours), RMSE (which is similar to MAE but penalizes large errors), and the coefficient of determination R^2.

We will thoroughly examine and compare the models, and then interpret them during the next sprint.

3 Review & Retrospective

3.1 Successes

We successfully completed all the tasks we assigned to the current sprint. A list of these completed tasks is presented in Table 2.

To ensure reproducibility, we created notebooks that allow you to execute all stages of the pipeline, including feature extraction, baseline model training, and the printing of initial metrics. The notebooks can be found in the folder **notebooks**.

Table 2. Sprint 1 issues.

Issue	Description
#46	Sprint 1 - Report
#34	Implement basic pipeline
#45	Implement data splitting
#44	Implement event log cleaning
#41	Refactor the pipeline to follow a strategy design pattern
#35	Baseline models
#40	Implement baseline remaining time prediction pipeline (regression)
#39	Implement baseline outcome prediction pipeline (classification)
#38	Extract log-based features from case prefixes
#37	Generate case prefixes from event log
#36	Preprocess event log
#30	Dataset

3.2 Difficulties

During the initial setup of the pipeline, we encountered several challenges:

- The initial pipeline setup did not allow for parallel development, which slowed down early progress. Primary reason was, that every implementation task depends on shared interfaces between each other, which are required to be defined before one can start work on them. We see this issue as unavoidable, but after defining and setting up the pipeline architecture, future sprints are not expected to be affected by this.
- The initial prefix generation implementation was inefficient and significantly slowed down feature extraction, which hindered testing and the implementation of log-based features and downstream pipeline stages. We improved

performance by replacing the original pandas dataframe slicing approach with NumPy trace arrays and prefix slice indices, achieving approximately a 3× speedup during feature extraction.

3.3 Task Tracking

Initially, the team used both Trello and GitHub Issues for sprint and task management. However, maintaining tasks across two separate platforms introduced unnecessary duplicate effort and reduced workflow efficiency.

GitHub Issues proved to be a better fit for our workflow due to its tight integration with pull requests and its support for linking issues, which allowed us to structure work hierarchically using parent and child issues. As a result, we discontinued the use of Trello and consolidated all task tracking, sprint planning, and issue management within GitHub Issues.

4 Moving Forward

4.1 Future Planning

Sprint 2 will focus on extending the current predictive process monitoring pipeline by incorporating time-series information and evaluating its impact on predictive performance. The planned work is divided into the following tracks:

- **Time-series incorporation.** We plan to align external time-series signals with the generated case prefixes. These signals include daily application arrivals, active-case counts, resource utilization, and approval rates over time.
- **Modeling and pipeline extension.** The existing pipeline will be extended to support hybrid feature sets combining traditional log-based features with aggregated time-series information. The modular architecture implemented in Sprint 1 will allow these additions without major restructuring.
- **Evaluation and reporting.** We will compare the extended models against the Sprint 1 baseline models to evaluate the impact of time-series features. The analysis will focus on predictive performance across different prefix lengths and on identifying the most influential features.

4.2 Role Allocation

We assigned the initial roles during the project setup phase which are listed in Table 3. Because the initial allocation was incomplete, we finalized it during the planning phase of Sprint 1 as listed in column **Sprint 1**.

For the next sprint we defined the new role allocation as listed in column **Sprint 2**.

Table 3. Role assignments for Sprints 1&2.

Role	Sprint 1	Sprint 2
Project Coordinator / Scrum Master	Michal	Konstantinos
Technical Lead / Architecture	Roman	Michal
Data & Preprocessing Lead	Roman, Konstantinos	Konstantinos
Modelling & Experimentation Lead	Michal	Roman
Evaluation & Visualization Lead	Konstantinos	Michal
Quality & Tooling (Tests, CI)	Konstantinos	Roman
Documentation & Reporting	Michał, Roman, Konstantinos (shared)	(stays shared)

References

1. van Dongen, B.F.: BPI Challenge 2017. Eindhoven University of Technology (2017). https://research.tue.nl/en/datasets/bpi-challenge-2017